

Date:
Student Name:
Student ID:

AI Workshop #9

Building AI Games

9.1. Introduction

In this workshop, we are going to learn how to build games using an artificial intelligence technique called **combinatorial search**.

In its most basic form, it can be thought of as a brute-force approach. We explore every single possible solution.

Later in the workshop, we will get smarter and find a way to short circuit the search and not have to try every single possibility.

We will learn how to use search algorithms to effectively come up with strategies to win a set of games.

We will then use these algorithms to build intelligent bots for different games.

By the end of this workshop, you will have a better understanding of these topics:

- Search algorithms in games
- Combinatorial search
- The Minimax algorithm
- Alpha-Beta pruning
- The Negamax algorithm
- Building a bot to play Last Coin Standing
- Building a bot to play Tic-Tac-Toe
- Building two bots to play Connect Four against each other
- Building two bots to play Hexapawn against each other

9.2 Using Search Algorithms in AI Games

Search algorithms are commonly used in games to figure out a strategy. The algorithms search through possible game moves and pick the best one.

There are various parameters to consider when implementing these searches – speed, accuracy, complexity, and so on.

These algorithms consider all possible game moves given a current game state and then evaluate each possible move to determine the best one.

The goal of these algorithms is to find the optimal move that will eventually lead to winning the game. Also, every game has a different set of rules and constraints. These algorithms consider these rules and constraints when exploring the best moves.

Games without opponents are easier to optimize than games with opponents. Gameplay becomes more complicated with games that have multiple players. Let's consider a two-player game. For every move made by a player to try to win the game, the opposing player will make a move to prevent the player from winning.

So, when a search algorithm finds the optimal set of moves from the current state, it cannot just make moves without considering the opposing player's countermoves. This means that search algorithms need to constantly be re-evaluated after each move.

Let's discuss how a computer perceives any given game. We can think of a game as a **search tree**. Each node in this tree represents a future state.

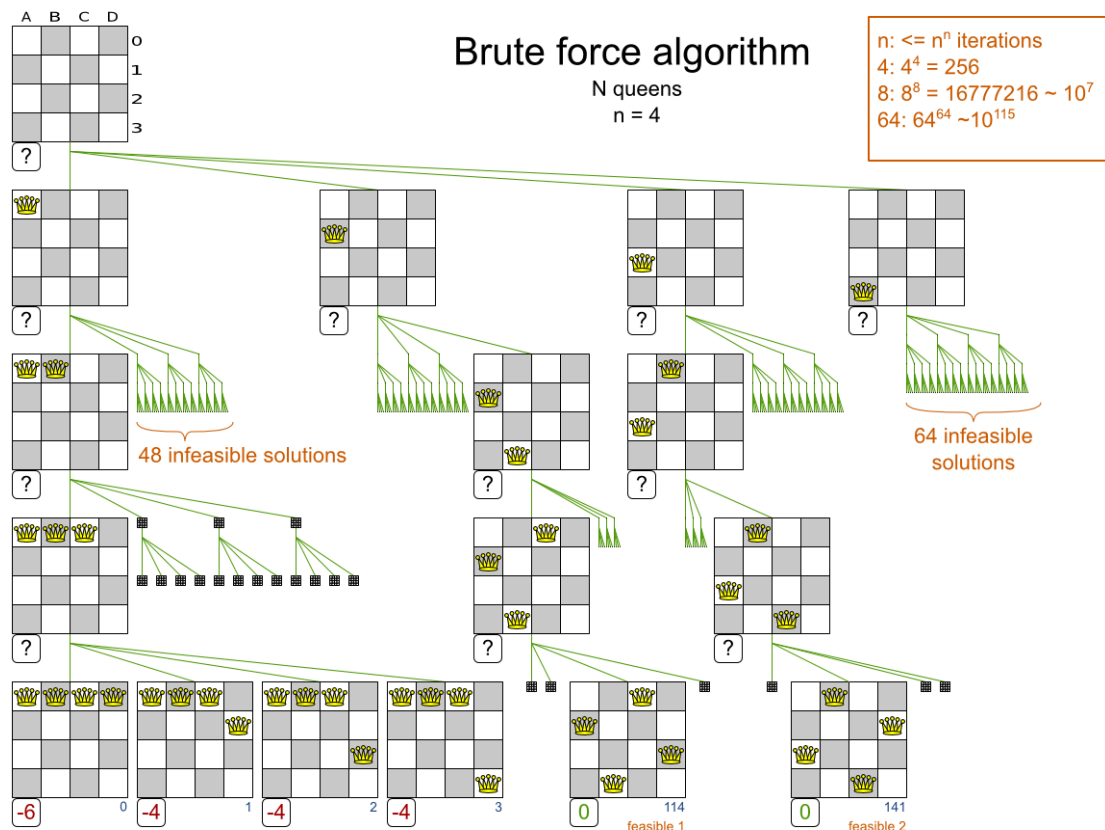
For example, if you are playing Tic-Tac-Toe (Noughts and Crosses), you can construct a tree to represent all possible moves. We start from the root of the tree, which is the starting point of the game. This node will have several children that represent various possible moves. Those children, in turn, will have more children that represent game states after more moves by the opponent. The terminal nodes of the tree represent the final moves of the game.

The game will either end in a draw or one of the players will win it. The search algorithms search through this tree to make decisions at each step of the game. We will now learn a variety of search techniques of how to include an exhaustive combinatorial search to help us to never lose at Tic-Tac-Toe and solve a variety of many other problems.

9.3 Combinatorial Search

Search algorithms appear to solve the problem of adding intelligence to games, but there's a drawback. These algorithms employ a type of search called **exhaustive search**, which is also known as a **brute-force search**. It basically explores the entire search space and tests every possible solution. It means that the algorithm will have to explore all the possible solutions before obtaining the optimal solution.

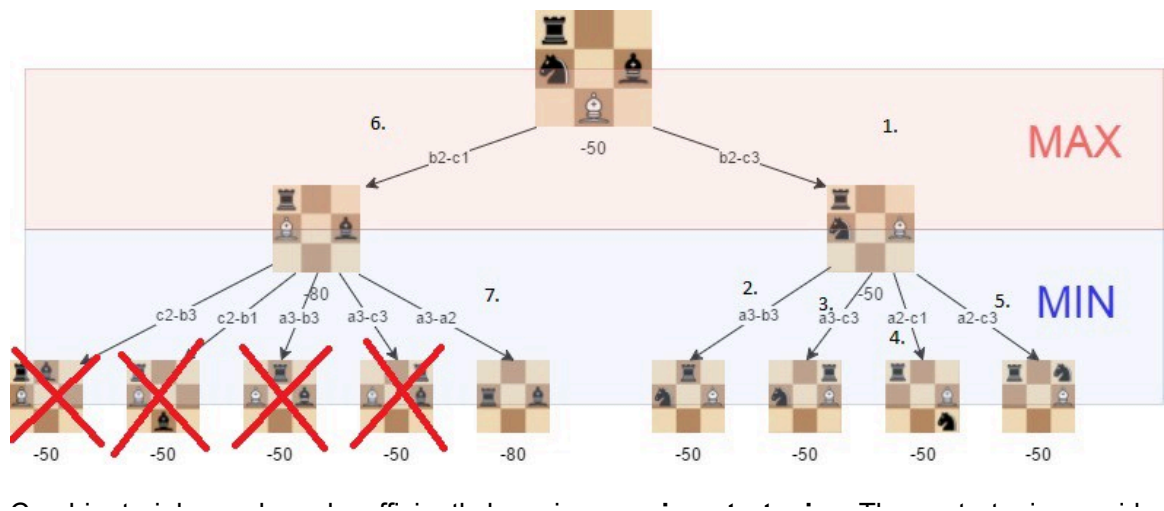
The following shows Chess playing with Brute-force search.



As the games get more complex, brute-force search may not be the best approach, because the number of possibilities becomes enormous. The search quickly becomes computationally intractable. In order to solve this problem, **combinatorial search** can be used to solve problems.

Combinatorial search refers to a field of study where search algorithms efficiently explore the solution space using heuristics to reduce the size of the search space. This is useful in games such as **Chess** or **Go**.

The following shows Combinational search use Minimax Algorithm on Chess playing.



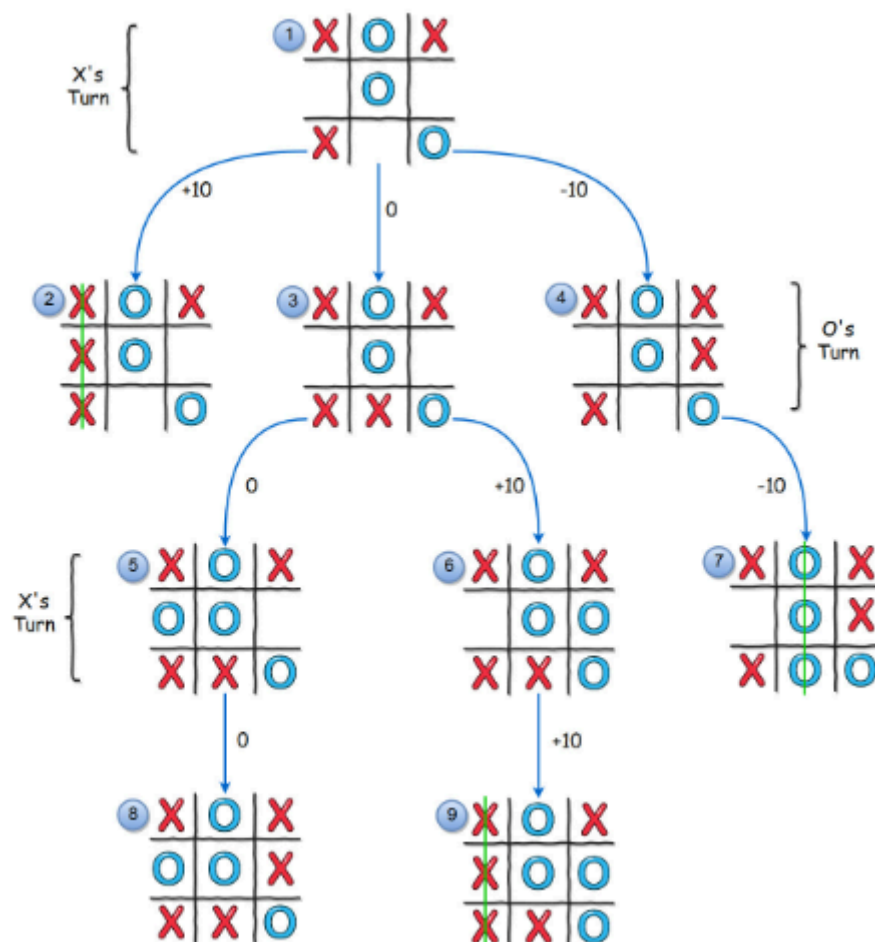
9.4 The MiniMax Algorithm

Now that we have briefly discussed combinatorial search in general, let's talk about the heuristics that are employed by combinatorial search algorithms.

These heuristics are used to speed up the search strategy and the **Minimax algorithm** is one such strategy used by combinatorial search.

When two players are playing against each other, their goals are diametrically opposed. Each player is trying to win. So, each side needs to predict what the opposing player is going to do in order to win the game. Keeping this in mind, Minimax tries to achieve this through strategy. **It will try to minimize the function that the opponent is trying to maximize.**

The following shows example of Tic-tac-toe playing using Minimax Algorithm.



As discussed earlier, brute force only works in simple games with a small number of possible moves. In more complicated cases, the computer cannot go through all the possible states to find the optimal gameplay.

In this case, the computer can try to calculate the optimal move based on the current state using a heuristic.

The computer constructs a tree and it starts from the bottom. It evaluates which moves would benefit its opponent.

The algorithm knows which moves the opponent is going to make based on the premise that the opponent will make the moves that would benefit them the most, and thereby be of the least benefit to the computer.

This outcome is one of the terminal nodes of the tree and the computer uses this position to work backwards. Each option that's available to the computer can be assigned a value and it can then pick the highest value to take an action.

9.5 Alpha-Beta Pruning

Minimax search is an efficient strategy, but it still ends up exploring parts of the tree that are irrelevant.

When an indicator on a node is found showing that a solution does not exist in that sub-tree, there is no need to evaluate that sub-tree. But Minimax search is too conservative, so it ends up exploring some of these sub-trees.

The **Alpha-Beta algorithm** is smarter and avoids searching parts of the tree that it discovers will not have the solution. This process is called **pruning** and **Alpha-Beta pruning** is a type of strategy that is used to avoid searching parts of the tree that do not contain the solution.

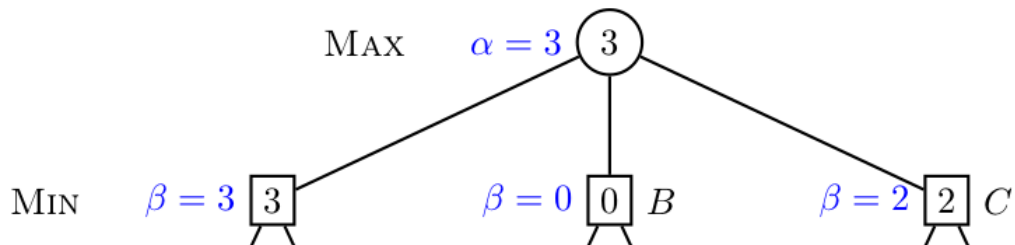
The Alpha and Beta parameters in Alpha-Beta pruning refer to the two bounds that are used during the calculation. These parameters refer to the values that restrict the set of possible solutions.

This is based on the section of the tree that has already been explored. **Alpha** is the maximum lower bound on the number of possible solutions and **Beta** is the minimum upper bound on the number of possible solutions.

As discussed earlier, each node can be assigned a value based on the current state.

When the algorithm considers any new node as a potential path to the solution, it can work out if the current estimate of the value of the node lies between Alpha and Beta. This is how it prunes the search.

The following shows an example of Minimax algorithm with alpha-beta pruning:



9.6 The Negamax Algorithm

The **Negamax algorithm** is a variant of Minimax that's frequently used in real-world implementations.

A two-player game is usually a **zero-sum game**, which means that one player's loss is equal to another player's gain and vice versa. Negamax uses this property extensively to come up with a strategy to increase its chances of winning the game.

In terms of a game, the value of a given position to the first player is the negation of the value to the second player.

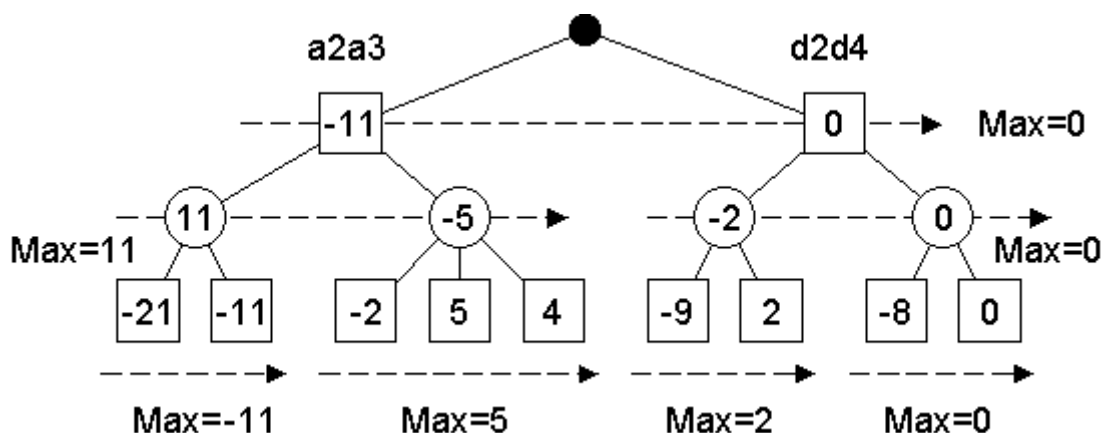
Each player looks for a move that will maximize the damage to the opponent. The value resulting from the move should be such that the opponent gets the least value.

This works both ways seamlessly, which means that a single method can be used to value the positions. This is where it has an advantage over Minimax in terms of simplicity.

Minimax requires that the first player select the move with the maximum value, whereas the second player must select a move with the minimum value.

Alpha-Beta pruning is used here as well.

The following shows an example of Negamax Algorithm.



Now that we've looked at a few of the most popular combinatorial search algorithms, let's install a library so that we can build some AI and see these algorithms in action.'

9.7 Installing the EasyAI Library

We will be using a library called easyAI in this workshop.

It is an artificial intelligence framework and it provides all the functionality necessary to build two-player games.

Install it by running the following command:

pip install easyAI

Some of the files need to be accessible in order to use some of the pre-built routines.

For ease of use, the code provided with this workshop contains a folder called easyAI.

Make sure you place this folder in the same folder as your code files.

This folder is basically a subset of the easyAI GitHub repository available here:

<https://github.com/Zulko/easyAI> (<https://github.com/Zulko/easyAI>)

You can go through the source code to make yourself more familiar with it.

9.8 Building a bot to play Last Coin Standing

In this game, there is a pile of coins and each player takes turns to take a number of coins from the pile.

There is a lower and an upper bound on the number of coins that can be taken from the pile.

The goal of the game is to avoid taking the last coin in the pile.

This recipe is a variant of the Game of Bones recipe given in the easyAI library.



Let's see how to build a game where the computer can play against the user.

1. Create a new Python file and import the following packages:

```
# This is a variant of the Game of Bones recipe given in the easyAI library

from easyAI import TwoPlayersGame, id_solve, Human_Player, AI_Player
from easyAI.AI import TT
```

2. Create a class to handle all the operations of the game. The code inherits from the base class `TwoPlayersGame` available in the `easyAI` library. There are a couple of parameters that must be defined for the code to function properly. The first one is the `players` variable. The player object will be discussed later. Create the class using the following code:

```
class LastCoinStanding(TwoPlayersGame):
    def __init__(self, players):
        # Define the players. Necessary parameter.
        self.players = players
```

3. Define the player that is going to start the game. The players are numbered starting with one. So, in this case, player one starts the game:

```
# Define who starts the game. Necessary parameter.
self.nplayer = 1
```

4. Define the number of coins in the pile. You are free to choose any number here. In this case, let's choose 25:

```
# Overall number of coins in the pile
self.num_coins = 25
```

5. Define the maximum number of coins that can be taken out in any move. You are free to choose any number for this parameter as well. Let's choose 4 in our case:

```
# Define max number of coins per move
self.max_coins = 4
```

6. Define all the possible moves. In this case, players can take either 1, 2, 3, or 4 coins in each move:

```
# Define possible moves
def possible_moves(self):
    return [str(x) for x in range(1, self.max_coins + 1)]
```

7. Define a method to remove the coins and keep track of the number of coins remaining in the pile

```
# Remove coins
```

```
def make_move(self, move):
    self.num_coins -= int(move)
```

8. Check if somebody won the game by checking the number of coins remaining:

```
# Did the opponent take the last coin?
def win(self):
    return self.num_coins <= 0
```

9. Stop the game after somebody wins it:

```
# Stop the game when somebody wins
def is_over(self):
    return self.win()
```

10. Compute the score based on the win method. It's necessary to define this method:

```
# Compute score
def scoring(self):
    return 100 if self.win() else 0
```

11. Define a method to show the current status of the p

```
# Show number of coins remaining in the pile
def show(self):
    print(self.num_coins, 'coins left in the pile')
```

12. Define the main function and start by defining the transposition table. Transposition tables are used in games to store the positions and movements to speed up the algorithm.

```
if __name__ == "__main__":
    # Define the transposition table
    tt = TT()
```

13. Define the method tentry to get the number of coins. It's an optional method that's used to create a string to describe the game:

```
# Define the method
LastCoinStanding.tentry = lambda self: self.num_coins
```

14. Let's solve the game using AI. The function id_solve is used to solve a given game using iterative deepening. It basically determines who can win a game using all the paths. It looks to answer questions such as:

- Can the first player force a win by playing perfectly?
- Will the computer always lose against a perfect opponent?

The method `id_solve` explores various options in the game's Negamax algorithm several times. It always starts at the initial state of the game and takes increasing depth to keep going. It will do so until the score indicates that somebody will win or lose. The second argument in the method takes a list of depths that it will try out. In this case, it will try all the

```
# Solve the game
result, depth, move = id_solve(LastCoinStanding,
                               range(2, 20), win_score=100, tt=tt)
print(result, depth, move)
```

15. Start the game against the computer:

```
# Start the game
game = LastCoinStanding([AI_Player(tt), Human_Player()])
game.play()
```

16. It's an interactive program, so it will expect input from the user. If you run the code, you will be basically playing against the computer. Your goal is to force the computer to take the last coin so that you win the game. If you run the code and try, you will see the following:

```
D:\CloudDrive\Workshops\AI Workshops\AIWorkshop9 - AI Games>python coins.py
d:2, a:0, m:1
d:3, a:0, m:1
d:4, a:0, m:1
d:5, a:0, m:1
d:6, a:0, m:1
d:7, a:0, m:1
d:8, a:0, m:1
d:9, a:0, m:1
d:10, a:100, m:4
1 10 4
```



Workshop 9.1 : Last Coin Standing Game

1. Combine all the above code and named it as **lastcoin.py**.
2. If everything is okay, you will start this interactive game. Play it, save and submit the code and result figure.
3. Without any good strategy, you will always loss. Why?
4. How to win that game? Any suggestion?
5. (Bonus) How can you make it more intelligent? Any idea? Submit your code and result figures.

9.9 Building a bot to play Tic-Tac-Toe

Tic-Tac-Toe (Noughts and Crosses) is perhaps one of the most famous games in the world.

Let's see how to build a game where the computer can play against the user.

This is a minor variant of the **Tic-Tac-Toe recipe** given in the **easyAI library**.



1. Create a new Python file and import the following packages:

```
# This is a variant of the Tic Tac Toe recipe given in the easyAI library

from easyAI import TwoPlayersGame, AI_Player, Negamax
from easyAI.Player import Human_Player
```

2. Define a class that contains all the methods to play the game. Start by defining the players and who starts the game:

```
class GameController(TwoPlayersGame):
    def __init__(self, players):
        # Define the players
        self.players = players

        # Define who starts the game
        self.nplayer = 1
```

3. We will be using a 3×3 board numbered from one to nine row-wise:

```
# Define the board
self.board = [0] * 9
```

4. Define a method to compute all the possible moves:

```
# Define possible moves
def possible_moves(self):
    return [a + 1 for a, b in enumerate(self.board) if b == 0]
```

5. Define a method to update the board after making a move:

```
# Make a move
def make_move(self, move):
    self.board[int(move) - 1] = self.nplayer
```

6. Define a method to see if somebody has lost the game. We will be checking if somebody has three in a row:

```
# Does the opponent have three in a line?
def loss_condition(self):
    possible_combinations = [[1,2,3], [4,5,6], [7,8,9],
                             [1,4,7], [2,5,8], [3,6,9], [1,5,9], [3,5,7]]
```

```

        return any([all([(self.board[i-1] == self.nopponent)
                        for i in combination]) for combination in
possible_combinations])

```

7. Check if the game is done using the `loss_condition` method:

```

# Check if the game is over
def is_over(self):
    return (self.possible_moves() == []) or self.loss_condition()

```

8. Define a method to show the current progress:

```

# Show current position
def show(self):
    print('\n'+'\n'.join([' '.join(['.', 'O', 'X'][self.board[3*j +
i]])
                        for i in range(3)]) for j in range(3)))

```

9. Compute the score using the `loss_condition` method:

```

# Compute the score
def scoring(self):
    return -100 if self.loss_condition() else 0

```

10. Define the main function and start by defining the algorithm. Negamax will be used as the AI algorithm for this game. The number of steps that the algorithm should think ahead can be specified in advance. In this case, let's choose 7:

```

if __name__ == "__main__":
    # Define the algorithm
    algorithm = Negamax(7)

```

11. Start the game:

```

# Start the game
GameController([Human_Player(), AI_Player(algorithm)]).play()

```

12. Just like the previous one. It's an interactive game where you play against the computer. If you run the code, you will initially get the following output. Try it, see whether you can win.

```

D:\CloudDrive\Workshops\AI Workshops\AIWorkshop9 - AI Games>python tic_tac_toe.py

. . .
. . .
. . .

Player 1 what do you play ? 1

Move #1: player 1 plays 1 :

0 . .
. . .
. . .

Move #2: player 2 plays 5 :

0 . .
. X .
. . .

Player 1 what do you play ? 2

Move #3: player 1 plays 2 :

0 0 .
. X .
. . .

Move #4: player 2 plays 3 :

0 0 X
. X .
. . .

Player 1 what do you play ? 7

Move #5: player 1 plays 7 :

```



Workshop 9.2 : Tic_Tac_Toe Game

1. Combine all the above code and named it as **tic_tac_toe.py**.
2. If everything is okay, you will start this interactive game. Play it, save and submit the code and result figure.
3. Without any good strategy, it is not easy to win the AI. Why?
4. To make it more challenging, expand the chess board to 5x5. One can win when they have got 4 piece in sequence. Save and submit it as **tic_tac_toe_4.py**.

9.10 Building two bots to play Connect Four™ against

each other



Connect Four™ is a popular two-player game sold under the Milton Bradley trademark.

It is also known by other names such as Four in a Row or Four Up.

In this game, the players take turns dropping discs into a vertical grid consisting of six rows and seven columns. The goal is to get four discs in a line.

This is a variant of the Connect Four recipe given in the easyAI library.

Let's see how to build it.

1. Create a new Python file and import the following packages:

```
# This is a variant of the Connect Four recipe given in the easyAI library

import numpy as np
from easyAI import TwoPlayersGame, Human_Player, AI_Player, \
    Negamax, SSS
```

2. Define a class that contains all the methods needed to play the game:

```
class GameController(TwoPlayersGame):
    def __init__(self, players, board = None):
        # Define the players
```

```
self.players = players
```

3. Define the board with six rows and seven columns:

```
# Define the configuration of the board
self.board = board if (board != None) else (
    np.array([[0 for i in range(7)] for j in range(6)]))
```

4. Define who's going to start the game. In this case, let's have player one start the game:

```
# Define who starts the game
self.nplayer = 1
```

5. Define the positions:

```
# Define the positions
self.pos_dir = np.array([[[i, 0], [0, 1]] for i in range(6)] +
    [[[0, i], [1, 0]] for i in range(7)] +
    [[[i, 0], [1, 1]] for i in range(1, 3)] +
    [[[0, i], [1, 1]] for i in range(4)] +
    [[[i, 6], [1, -1]] for i in range(1, 3)] +
    [[[0, i], [1, -1]] for i in range(3, 7)])
```

6. Define a method to get all the possible moves:

```
# Define possible moves
def possible_moves(self):
    return [i for i in range(7) if (self.board[:, i].min() == 0)]
```

7. Define a method to control how to make a move:

```
# Define how to make the move
def make_move(self, column):
    line = np.argmin(self.board[:, column] != 0)
    self.board[line, column] = self.nplayer
```

8. Define a method to show the current status:

```
# Show the current status
def show(self):
    print('\n' + '\n'.join(
        ['0 1 2 3 4 5 6', 13 * '-'] +
        [' '.join(['.', 'O', 'X'][self.board[5 - j][i]])
         for i in range(7)]) for j in range(6)))
```

9. Define a method to compute what a loss looks like. Whenever somebody gets four in a line, that player wins the game:

```
# Define what a loss_condition looks like
```

```

def loss_condition(self):
    for pos, direction in self.pos_dir:
        streak = 0
        while (0 <= pos[0] <= 5) and (0 <= pos[1] <= 6):
            if self.board[pos[0], pos[1]] == self.nopponent:
                streak += 1
                if streak == 4:
                    return True
            else:
                streak = 0

        pos = pos + direction

    return False

```

10. Check if the game is over by using the `loss_condition` method:

```

# Check if the game is over
def is_over(self):
    return (self.board.min() > 0) or self.loss_condition()

```

11. Compute the score:

```

# Compute the score
def scoring(self):
    return -100 if self.loss_condition() else 0

```

12. Define the main function and start by defining the algorithms. Then, the two algorithms will play against each other. Negamax will be used for the first computer player and the SSS* algorithm for the second computer player. SSS* is a search algorithm that conducts a state space search by traversing the tree in a best-first style. Both methods take, as an input argument, the number of turns in advance to think about. In this case, let's use 5 for both:

```

if __name__ == '__main__':
    # Define the algorithms that will be used
    algo_neg = Negamax(5)
    algo_sss = SSS(5)

```

13. Start playing the game:

```

# Start the game
game = GameController([AI_Player(algo_neg), AI_Player(algo_sss)])
game.play()

```

14. Print the result:

```

# Print the result
if game.loss_condition():
    print('\nPlayer', game.nopponent, 'wins.')
else:

```

```
print("\nIt's a draw.")
```



Noted that, this is not an interactive game. The code pits one algorithm against another. The Negamax algorithm is player one and the SSS* algorithm is player two.

15. If you run the code, you will initially get the following output:

```
D:\CloudDrive\Workshops\AI Workshops\AIWorkshop9 - AI Games>python connect_four.py
0 1 2 3 4 5 6
-----
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .

Move #1: player 1 plays 0 :
0 1 2 3 4 5 6
-----
. . . . .
. . . . .
. . . . .
. . . . .
0 . . . . .

Move #2: player 2 plays 0 :
0 1 2 3 4 5 6
-----
. . . . .
. . . . .
. . . . .
X . . . . .
0 . . . . .

Move #3: player 1 plays 0 :
0 1 2 3 4 5 6
-----
. . . . .
. . . . .
0 . . . . .
X . . . . .
0 . . . . .
```

16. If you scroll down, you will see the following toward the end:

```

0 1 2 3 4 5 6
-----
X X 0 0 X . .
0 0 X X 0 . .
X X 0 0 X . .
0 0 X X 0 0 .
X X 0 X X X .
0 0 0 X 0 0 .

```

Move #34: player 2 plays 5 :

```

0 1 2 3 4 5 6
-----
X X 0 0 X . .
0 0 X X 0 . .
X X 0 0 X X .
0 0 X X 0 0 .
X X 0 X X X .
0 0 0 X 0 0 .

```

Move #35: player 1 plays 6 :

```

0 1 2 3 4 5 6
-----
X X 0 0 X . .
0 0 X X 0 . .
X X 0 0 X X .
0 0 X X 0 0 .
X X 0 X X X .
0 0 0 X 0 0 0

```

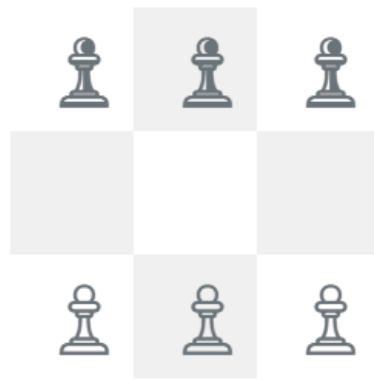


Workshop 9.3 : Connect_FOUR Game

1. Combine all the above code and named it as **connect4_M2M.py**.
2. If everything is okay, you should see the above game results of the two player bots, save and submit the code and result figure.
3. Change this game into an interactive game between Human and Bot with 2 level of challenges:
 - Bot with Negamax Algorithm (connect4_M2H_Negamax.py)
 - Bot with SSS"* Algorithm (connect4_M2H_SSS.py)
4. (Bonus) Design one more algorithm to beat these two Bots. Submit your code and result figures.

9.11 Building two bots to play Hexapawn against each

other



0 / 0 victories

How to play

- You play white, the AI plays black
- White always starts
- On your turn, there are two types of valid moves:
 - i. Move one of your pawns 1 square straight ahead if free
 - ii. Move one of your pawns 1 square diagonally ahead if occupied by the opponent and capture the piece
- The game is won in one of three ways:
 - i. Moving a pawn to the opposite end of the board
 - ii. Capturing all enemy pawns
 - iii. Achieving a position in which the enemy can't move

Hexapawn is a two-player game starting with a chessboard of size $N \times M$.

Pawns exist on each side of the board and the goal is to advance a pawn all the way to the other end of the board.

The standard pawn rules of chess apply.

This is a variant of the Hexapawn recipe given in the easyAI library.

Two bots will be created and pitted against each other.

1. Create a new Python file and import the following packages:

```
# This is a variant of the Hexapawn recipe given in the easyAI library

from easyAI import TwoPlayersGame, AI_Player, \
    Human_Player, Negamax
```

2. Define a class that contains all the methods necessary to control the game. Start by defining the number of pawns on each side and the length of the board. Create a list of tuples containing the positions:

```
class GameController(TwoPlayersGame):
    def __init__(self, players, size = (4, 4)):
        self.size = size
        num_pawns, len_board = size
        p = [(i, j) for j in range(len_board)] \
            for i in [0, num_pawns - 1]
```

3. Assign the direction, goals, and pawns to each player:

```
for i, d, goal, pawns in [(0, 1, num_pawns - 1,
    p[0]), (1, -1, 0, p[1])]:
    players[i].direction = d
    players[i].goal_line = goal
```

```
players[i].pawns = pawns
```

4. Define the players and specify who starts first:

```
# Define the players
self.players = players

# Define who starts first
self.nplayer = 1
```

5. Define the alphabets that will be used to identify positions such as B6 or C7 on a chessboard:

```
# Define the alphabets
self.alphabets = 'ABCDEFGHIJ'
```

6. Define a lambda function to convert the strings to tuples:

```
# Convert B4 to (1, 3)
self.to_tuple = lambda s: (self.alphabets.index(s[0]),
                           int(s[1:]) - 1)
```

7. Define a lambda function to convert the tuples to strings:

```
# Convert (1, 3) to B4
self.to_string = lambda move: ' '.join([self.alphabets[
    move[i][0]] + str(move[i][1] + 1)
    for i in (0, 1)])
```

8. Define a method to compute the possible moves:

```
# Define the possible moves
def possible_moves(self):
    moves = []
    opponent_pawns = self.opponent.pawns
    d = self.player.direction
```

9. If you don't find an opponent pawn in a position, then that's a valid move:

```
for i, j in self.player.pawns:
    if (i + d, j) not in opponent_pawns:
        moves.append(((i, j), (i + d, j)))

    if (i + d, j + 1) in opponent_pawns:
        moves.append(((i, j), (i + d, j + 1)))

    if (i + d, j - 1) in opponent_pawns:
        moves.append(((i, j), (i + d, j - 1)))
```

```
return list(map(self.to_string, [(i, j) for i, j in moves]))
```

10. Define how to make a move and update the pawns based on that:

```
# Define how to make a move
def make_move(self, move):
    move = list(map(self.to_tuple, move.split(' ')))
    ind = self.player.pawns.index(move[0])
    self.player.pawns[ind] = move[1]

    if move[1] in self.opponent.pawns:
        self.opponent.pawns.remove(move[1])
```

11. Define the conditions for a loss. If a player gets 4 in a line, then the opponent loses:

```
# Define what a loss looks like
def loss_condition(self):
    return (any([i == self.opponent.goal_line
                for i, j in self.opponent.pawns])
            or (self.possible_moves() == []))
```

12. Check if the game is done using the loss_condition method:

```
# Check if the game is over
def is_over(self):
    return self.loss_condition()
```

13. Print the current status:

```
# Show the current status
def show(self):
    f = lambda x: '1' if x in self.players[0].pawns else (
        '2' if x in self.players[1].pawns else '.')

    print("\n".join([" ".join([f((i, j))
                              for j in range(self.size[1])])
                    for i in range(self.size[0])]))
```

14. Define the main function and start by defining the scoring lambda function:

```
if __name__ == '__main__':
    # Compute the score
    scoring = lambda game: -100 if game.loss_condition() else 0
```

15. Define the algorithm to be used. In this case, we will use Negamax, which can calculate 12 moves in advance and uses a scoring lambda function for strategy:

```
# Define the algorithm
```

```
algorithm = Negamax(12, scoring)
```

16. Start playing the game:

```
# Start the game
game = GameController([AI_Player(algorithm),
                       AI_Player(algorithm)])
game.play()
print('\nPlayer', game.nopponent, 'wins after', game.nmove , 'turns')
```



Noted that it's not an interactive game. Two bots are created and pitted against each other.

17. If you run the code, you will see the following results. Player1 wins after 8 turns.

```
D:\CloudDrive\Workshops\AI Workshops\AIWorkshop9 - AI Games>python hexapawn.py  
1 1 1 1  
.  
.  
.
```



Workshop 9.4 : Hexapawn Game

1. Combine all the above code and named it as **hexapawn_M2M.py**.
2. If everything is okay, you should see the above game results of the two player bots, save and submit the code and result figure.
3. Modify this game with Player 1 using Negamax and Player 2 using SSS algorithm and save as **hexapawn_M2M_2.py**. Who wins?
4. Change this game into an interactive game between Human and Bot with 2 level of challenges:
 - Bot with Negamax Algorithm (hexapawn_M2H_Negamax.py)
 - Bot with SSS* Algorithm (hexapawn_M2H_SSS.py)
4. (Bonus) Design one more algorithm to beat these two Bots. Submit your code and result figures.

9.12 Summary

In this workshop, we discussed how to build games using a special type of artificial intelligence technique called combinatorial search.

We learned how to use these types of search algorithms to effectively come up with strategies to win games.

These algorithms can be used to build game playing machines for more complicated games as well as solve a wide variety of problems.

We talked about combinatorial search and how it can be used to speed up the search process.

We learned about Minimax and Alpha-Beta pruning.

We learned how the Negamax algorithm is used in practice.

We then used these algorithms to build bots to play Last Coin Standing and Tic-Tac-Toe.

We learned how to build two bots to play against each other in Connect Four and Hexapawn.



Group Project: Building Your AI Chess Bot

So far, we have learnt:

- Different basic types of AI-based search algorithms for winning in a chess game such as Combinational search, Minimax algorithm, Alpha-beta pruning, Negamax algorithm, SSS* algorithm, etc.
- Implementation of various basic AI chess games such as Tic-Tac-Toe, Connect-FOUR and Hexapawn.

In the Group Project, form a group to design and implement an AI-based Chess Bot with:

- At least 3 levels of AI challenges, one above the others
- Build both interactive H2M and M2M - non-interactive AI Chess Game to demonstrate Chess Bot in various levels of challenge.
- Highly recommend to add your own AI algorithm to beat the classical methods
- Extra marks will be given with nice looking interface.



Project Term Paper

As part of the project deliverable, each project group have to submit a Project Term Paper to discuss the literature review of AI-game/AI-Chess bot, related technology and their proposed AI Chess Bot with experimental results/ simulation results.

The TOC (Table of Content) include:

- Project Title
- Abstract (200 - 250 words)
- Keywords (4 - 6)
- Introduction
- Literature Review
- Methodology
- Implementation
- Experimental Results
- Performance Analysis
- Future Work
- Conclusion
- Reference

Note:

- Total no. of page: 8 - 12 pages, double-column, single-spacing
- Paper format: Please refer to the attached IEEE Paper Template in the directory folder.
- Similarity: Less than 20%

In []: